

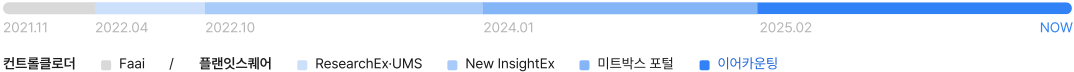
타입·상태·렌더링 경계를 설계하는 프론트엔드 엔지니어

React·TypeScript 기반 5년차. 회계·세무 SaaS, B2B 서비스, 데이터 시각화 제품을 개발했습니다.
서버 상태와 클라이언트 상태를 분리하고, 캐시 무효화 범위·리렌더 경계·의존성 방향을 설계 기준으로 삼습니다. API 타입은 서버 스키마에서 생성하고 런타임 경계는 스키마 검증으로 막습니다.
성능은 값이 아니라 병목을 특정한 뒤 메모이제이션·구독 범위 축소로 해결합니다.

Career Summary

회계·세무 SaaS의 프론트엔드를 주도하며 저장소 커밋 1,563건을 담당했고, 상태 정합성이 완성도를 좌우하는 도메인을 다뤘습니다.
반복 입력은 배치 처리로, 외부 시스템 결과는 실시간 상태로 전환해 ‘실패하면 다시 하기’의 비용을 구조적으로 줄였습니다.
화면에 잘못된 값이 노출되자 프론트엔드 구현만 확인하는 데 그치지 않고, 기획서·법령·실제 코드를 대조해 세무 로직의 오류까지 추적했습니다.
타입·테스트·리뷰 기준을 G 검증으로 옮겨 AI 페어링으로 늘어난 코드에서도 품질이 개인 판단에 의존하지 않게 만들었습니다.

Experience 2021.11 – NOW · 5년차



이여카운팅 SAP ERP 연동 회계·세무 SaaS 2025.02 – Present

역할 · Frontend Lead Contributor
기술 · React 19, TypeScript, Vite, TanStack Query, Jotai, Mantine, Tailwind CSS, Ky, openapi-typescript, Zod, Vitest, MSW, Playwright

개인 사용자·재무 담당자·관리자가 다른 권한으로 같은 데이터를 다뤄, 상태 정합성이 완성도를 좌우합니다.

저장소 전체 커밋 중 1,563개를 담당했습니다. 프론트엔드를 주도하면서 제품 완성에 필요한 문제는 역할의 경계에 한정하지 않고 API, 서버 로직, DB, 배포 인프라까지 직접 개발했습니다.

배치 처리 파이프라인과 결재선 도메인 모델 설계

재무 담당자는 비슷한 데이터를 수십~수백 건씩 처리하며 건마다 선택→입력→저장을 반복했습니다.

결재 상태와 SAP 전송 상태가 한 필드에 뒤섞여 어느 단계에서 막혔는지 구분할 수 없었고,

결재 후 환불·비용계정·세금코드를 재무팀이 검토하는 단계는 시스템에 없었습니다.

사용자 조사로 실제 조회 순서를 파악한 뒤 화면 구조와 결재선 도메인 모델을 직접 설계했습니다.

설계 결정

- 저장 전 optimistic 반영은 배제 — SAP 전송이 결린 회계 데이터라 미확정 값을 먼저 그리지 않음
- 대신 서버 확정 후 refetch 도착까지의 공백을 행 단위 오버레이 atom으로 보정
- 기간 조건을 쿼리 키에 포함해 프라셋 전환 시 전체 재조회 대신 캐시 재사용
- 결재 상태와 SAP 전송 상태를 각각 서버 필드로 두고 표시단계는 클라이언트 파생값으로 계산
- 결제선을 단계 수 고정 배열이 아니라 (전표유형 × 금액구간 × 차수) 매칭 규칙으로 모델링

구현

- 그리드 검증·payload 빌드·배치 실행을 useBatchWrite 제네릭 훅으로 추출
- 세금계산서 도메인 추가 시 드러난 중복을 주입 지점으로 분리 — 하위 혹은 mutation과 payload만 교체
- 편집모드는 useBulkEdit으로 원본 스냅샷과 변경 행 diff만 보관 — 전체 폼 상태 미보유
- 부분 실패를 행별 result(success·errorCode·message)로 받아 실패 행만 재전송 가능 상태 유지
- 같은 차수 병렬 결재와 N-of-M 정족수 판정을 결재자별 상태와 문서 상태로 분리 계산

SSE 이벤트 기반 비동기 작업 상태 동기화

결제가 끝난 전표는 SAP에서 비동기로 생성되지만 클라이언트는 그 결과를 알 수 없었습니다.

동기 호출도 15초를 넘기면 클라이언트만 timeout돼, 사용자는 실패로 오해하고 재실행했습니다.

요청 수락(202)과 작업 완료를 별도 상태로 분리하고 서버 상태를 푸시로 받도록 설계했습니다.

polling 대신 SSE로 수신하고, 이벤트를 서버 상태 캐시에 반영해 화면이 스스로 갱신되게 했습니다

설계 결정

- 화면을 지켜보지 않아도 진행·성공·실패가 앱 전역 토스트로 전달
- 요청 수락과 처리 결과를 구분해 timeout을 실패로 오인한 재실행 방지
- 마지막 동기화 시작·처리 건수·미동기화 건수를 제공해 지금 실행이 필요한지 판단
- 처리 중에는 동일 액션 버튼을 disabled 처리해 중복 요청을 클라이언트에서 차단
- 상태 확인 후 SAP 전표 상세로 이어지는 동선을 연결해 다음 행동까지 안내

구현

- 혹마다 각자 EventSource를 열던 구조를 참조 카운팅 싱글턴으로 통합 — 사용자당 커넥션 2개 → 1개
- 연결·재연결(3초 간격 최대 5회, onopen에 카운트 리셋)-정리를 한 계층에 일원화
- 소비 혹은 이벤트명·payload 타임만 주입받는 래퍼로 축소(110→81줄, 62→12줄), 호출부 시그니처 불변
- 이벤트 수신 시 영향받는 쿼리 키만 선별 무효화해 전체 refetch와 불필요한 리랜더 억제
- EventSource stub과 fake timer로 커넥션 공유·부분 언마운트·재연결 상한을 테스트 16건으로 고정

■ SAP로 나간 전표의 복구 흐름 설계

SAP로 전송된 전표는 잘못돼도 화면에서 되돌릴 방법이 없어 따로 문의해야 설계였습니다.

전송 상태를 한 화면에 모으고, 상태별로 실제 가능한 복구 동작만 열어주도록 설계했습니다.

되돌리기는 사유와 요약을 다시 확인한 뒤 실행하도록 2단계로 나눠 오조작을 막았습니다.

설계 결정

- 전송중·완료·역분개됨·실패·이력없음을 범지로 구분하고 실패 사유를 함께 표시
- 상태에 따라 재전송과 역분개 중 실제 가능한 동작만 활성화
- 마감 기간이면 SAP가 거부하므로 해당 사유 선택 시 전기일을 필수 입력으로 요구
- SAP 거절 응답 메시지를 그대로 노출해 사용자가 원인을 바로 확인

구현

- 전송 이력 판별자로 버튼 활성화 조건 정의 (재전송은 정분개 실패만, 역분개는 전송 완료건)
- 사유 선택과 상세 입력을 분리하고 요약 재확인 후 실행하는 2단계 모달 구성
- 완료 전표 큐 API·타입과 라우트·메뉴·다국어까지 함께 구성

■ 기획서·법령·코드 3자 대조로 세무 결함 추적

세무 비전문가가 구현한 원천세 로직이라 기획서와 코드 중 어느 쪽이 맞는지 아무도 모르는 상태였습니다.

계산은 서버 영역이지만 틀린 값이 화면으로 나가면 같은 문제라, 담당 범위를 넘어 직접 대조했습니다.

항목별로 '무엇이 다른가 → 누가 틀렸나 → 무엇을 할 것인가'를 추적하고 근거를 위키에 남겼습니다.

대조 방식

- 11개 항목을 전수 대조해 코드 결함 5건·기획서 오류 2건을 분리 식별
- 조문 해석이 갈리는 항목은 LLM 위키에 근거를 모아 판단하고 이후 재사용하도록 보존
- 기획서의 '소득금액×0.1' 표기가 오타임을 확인 — 그대로면 지방세가 10배 과다
- '1원 절사 vs 10원 절사' 충돌은 단계별 규정(과세표준·납부세액)임을 밝혀 모순 해소
- 시마다 답이 갈리던 소액부징수 기준(소득세 1,000원·지방세 100원)을 조문으로 확정
- 코드 주석이 '지방세 1,000원'으로 뒤바뀌어 있던 것을 찾아 상수값 기준으로 교정

확인된 결함

- 대조 결과를 항목별 차이 로그로 관리 — 정상 항목도 남겨 '안 본 것'과 구분
- 세액 계산이 3개 진입 경로를 공유하는 구조를 파악해 수정 지점을 한 곳으로 좁혀 전달
- 소액부징수 게이트가 소득유형을 자기판별하도록 수정 방향을 제시하고 경계값 테스트로 고정되게 함
- 2024.7.1 개정으로 폐지된 사업소득 소액부징수를 찾아 적용 대상에서 제외

■ 타입 생성·런타임 검증·CI 게이트로 API 계약 고정

프론트엔드를 단독으로 담당해 타입·테스트·리뷰 기준이 개인 판단에 남아 있었습니다.

anya가 많이 손으로 하나씩 타입을 붙였지만, 서버 스펙이 바뀔 때마다 같은 불일치가 다시 생겼고

고치는 속도보다 어긋나는 속도가 빨랐습니다. 개별 수정 대신 타입의 출처를 바꾸기로 했습니다.

기준도 문서로 남기는 대신 CI가 검증하는 형태로 이동했습니다.

설계 결정

- 수작업 API 타입을 서버 스키마 생성물로 대체 — 불일치를 런타임에서 빌드타임으로 이동
- 생성물(31,918줄)은 커밋하되 수정 불가로 취급 — CI가 재생성 후 diff로 검사
- openapi.json을 키 정렬로 정규화 — 순서만 바뀐 diff가 drift 게이트를 무의미하게 만들어서
- Zod 전면 도입은 배제 — 스키마가 있는 경계는 생성 타임으로 충분, BE가 안 타이핑한 경계에만 적용
- 검증 정책을 헬퍼 주석에 명시해 다음 사람이 매번 판단하지 않게 고정

구현

- openapi-typescript로 schema.d.ts 생성, gen:api가 스펙 정규화까지 수행
- CI: 재생성 후 git diff --exit-code — 생성물이 커밋본과 다르면 실패
- parseResponse·safeParseResponse로 런타임 검증 지점을 11개 경계에 집약
- Vitest unit 39 · MSW integration 6 · Playwright e2e 18로 계층 분리

구조와 협업

- PR마다 다중 에이전트 코드리뷰를 직접 실행해 반복 점검 항목을 사람 눈에서 덜어냄
- FSD 기반 features → entities → shared 단방향 의존성 규칙 정의 및 점진 적용
- refresh 토큰을 HttpOnly 쿠키로 전환해 프론트에서 토큰을 제거하고 라우팅 가드를 E2E로 고정
- 세션 설계 구현 위치를 함께 탐색하고 근거 부족한 답변은 검증 대상으로 분류하는 LLM 위키 구축
- 이슈 생성 시 트래킹 Excel에 항목·문서 링크가 자동 기록되도록 연결
- health check 실패 시 이전 버전으로 롤백하는 배포 흐름 구성
- 디자인 토큰 우회를 막는 hex 하드코딩 CI 게이트 추가

미트박스 포털 축산 파트너용 B2B 이커머스 구독형 SaaS 2024.01 ~ 2025.01

역할 · 포털·관리자·모바일 프론트엔드 단독 구축

기술 · React 18, TypeScript, Webpack 5, Ant Design 5, Recoil, TanStack Query, Firebase FCM, React Router v6, i18next

축산 파트너용 포털, 운영자 관리자, 웹뷰 기반 모바일 앱으로 구성된 제품입니다.

세 개 앱을 혼자 담당해, 같은 코드를 세 번 쓰지 않는 것이 최대 관심사였습니다.

인증과 권한을 다른 축으로 분리한 라우팅 가드

사용자와 관리자의 진입점·권한이 다르고 2FA·SMS 인증까지 겹쳐, 한 가드에 넣으면 조건문이 계속 늘 구조였습니다.

그래서 처음부터 '로그인했는가'와 '권한이 있는가'를 별도 계층으로 나눴습니다.

- 토큰 기반 비밀번호 재설정과 3시간 자동 로그아웃·세션 갱신 구성
- 인증을 확인하는 Private Route와 권한을 확인하는 Permission Route 분리
- 주요 로그인 시나리오를 Jest와 Testing Library로 테스트

CRA eject 후 세 개 앱이 공유하는 빌드 기반 구성

초기 환경이 CRA로 정해져 있었고, 이미 쌓인 것이 많아 다른 틀체인으로 옮기는 선택지가 없었습니다.

- eject로 webpack-babel 설정을 드러내 포털·관리자가 빌드 설정과 공통 모듈을 공유
- TypeScript·SCSS·SVG 컴포넌트·정적 리소스 처리를 직접 구성
- Fast Refresh와 개발 서버 설정, 빌드 단계 타임 검사 편입
- eject는 되돌릴 수 없으므로 설정 변경을 최소한으로 유지하고 구조 공유에만 사용
- 반복되는 테이블·검색·폼·모달 패턴을 공통화해 세 앱이 재사용

공통 컴포넌트 — UI 책임만 지고 도메인은 조합으로

테이블·검색·폼·모달은 포털과 관리자에 반복되지만 화면별 요구가 조금씩 달랐습니다.

컴포넌트 내부 분기로 흡수하지 않고 UI 책임과 도메인 책임을 나눠 조합으로 처리했습니다.

웹 푸시와 웹뷰 모바일

Firebase Cloud Messaging을 활용해 포그라운드, 백그라운드, 서비스 워커 환경의 웹 푸시를 구현했습니다.

토큰 발급부터 백그라운드·서비스 워커까지 전체 흐름을 구현하고 기술 문서 3부작으로 정리했습니다.

모바일은 웹뷰로 구성해 화면을 재사용하고, ChatGPT 챗봇과 STT 음성 입력을 통합했습니다.

추가 구현

- i18next 기반 다국어 처리
- 공지·FAQ 팝업·회원·요금제 등 운영자 관리 기능과 파일 업로드·변경 이력 화면

New InsightEx 병원 실적 관리 대시보드 2022.10 ~ 2023.12

역할 · 개발 환경 구축, 레거시 리팩터링, 데이터 시각화

기술 · React, TypeScript, ECharts, DevExtreme DataGrid, PivotGrid

병원 실적·운영 데이터를 분석하는 대시보드로, 레거시 구조를 정리하고 개발 환경과 화면 구조를 재구성했습니다.

- 프론트엔드 개발 환경을 처음부터 구축
- ECharts 기반 병원 실적 및 운영 데이터 시각화
- DevExtreme DataGrid·PivotGrid 기반 대용량 데이터 조회 화면 구현
- 합계, 사용자 정의 정렬, 다중 필터, 무한 스크롤 구현

차트를 그리기 전에 사용자가 비교·판단할 것을 먼저 파악해 데이터 구조와 시각화 방식을 정했습니다.

ResearchEx · UMS 임상 데이터 검색 웹 · 사내 소통 게시판 2022.04 ~ 2022.09

기술 · React, TypeScript, SQL Editor, HTML Editor, SSO

- SQL Editor 기반 임상 데이터 탐색 화면과 소아 성장 시계열 차트 구현
- HTML Editor 기반 게시판 CRUD 구현과 SSO 사용자 정보 연동
- 사용자 권한에 따른 라우팅과 화면 액션 분기 처리

컨트롤클로더

Frontend Developer2021.11 – 2022.04

프로젝트 · Faai 웹앱·운영관리 웹
기술 · Vue.js, Vuex, Quasar, Capacitor, AWS CloudFront, S3

사내 운영관리 웹과 고객용 웹앱의 특정 기능을 함께 개발하며 두 사용자 그룹의 화면을 담당했습니다.

제작 수량 화면을 현장이 익숙한 테이블 UI로 바꿔 조회 흐름을 단순화했습니다.

배포 · 인프라

- DEV, QA, PRODUCTION 3단 환경 분리 배포 파이프라인 구성
- AWS CloudFront-S3 기반 정적 웹 배포와 Capacitor Android APK 빌드 배포
- 기능별로 배포 단계를 나눠 안정화된 기능부터 순차 운영 환경에 반영

화면 개발

- Vuex 기반 전역 상태 관리와 계층형 트리 구조 CRUD 구현
- 드래그로 우선순위를 변경해 DB에 반영하는 정렬 기능 구현
- 검색 조건을 RESTful API 파라미터로 전달해 필요한 데이터만 조회

그 외 경험

프리랜스 · 인턴으로 참여한 프로젝트

SAGEON

Frontend Freelance2025.06 – 2025.09

프로젝트 · Metaflux 보험 계산기 UI 리뉴얼
기술 · React, TypeScript, Context API

보험료·준비금 계산 제품에서 계산기 화면의 렌더링과 인터랙션 리뉴얼을 담당했습니다.

입력값이 결과에 즉시 반영되지 않는 병목을 추적해 상위 Context Provider의 value 재생성으로 특정했고,

코어 로직과 Provider 구조는 변경 불가라 화면 레이아웃에서 memo-useMemo·구독 분리로 리렌더 범위를 줄였습니다.

구조 변경이 필요한 구간은 원인과 개선안을 리포트로 정리해 전달했습니다.

퀀텀에이아이

Frontend Intern2021.07 – 2021.08

React Native 기반 지도 맛집 추천 앱 Road Pick 베타 개발에 참여해 지도·위치 화면과 인터랙션을 구현했습니다.

Skills

Frontend	React, TypeScript, JavaScript, React Native, Vue.js
State & Data	TanStack Query (React-Query), Recoil, Jotai, React Hook Form, Zod
Styling & Build	Emotion, Tailwind CSS, Webpack 5, Vite
Testing & CI	Vitest, Jest, Testing Library, MSW, Playwright, GitHub Actions

Technical Writing

실무 문제와 기술 선택의 근거를 블로그에 기록합니다. 원인 추적 과정과 대안의 트레이드오프 중심으로 씁니다.

주요 주제

- Firestore Cloud Messaging 웹 푸시 구현 3부작
- Concurrent 기능 적용 과정
- React Compiler 도입 검토
- ESLint와 useEffect 사용 기준

Education & Certifications

Wanted × WeCode Pre-Onboarding 프론트엔드 과정 수료	2021.07 – 2021.09
WeCode Bootcamp 프론트엔드 과정 수료	2021.05 – 2021.07
정보처리기사	2020.07
웹디자인개발가능사 (과정평가형)	2020.03